In [1]:
```
# a. Set up the environment
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Dropout, GlobalAveragePooling2
from tensorflow.keras.applications import VGG16

# b. Loading and preprocessing data

# Dataset folder path
data_dir = 'fashion mnist'

# Fashion MNIST class names
class_names = [
    'Ankle-boot', 'Bag', 'Coat', 'Dress', 'Pullover',
    'Sandal', 'Shirt', 'Sneaker', 'T-shirt-top', 'Trouser'
]

# Load training data
train_ds = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    class_names=class_names,
    validation_split=0.2,
    subset='training',
    seed=42,
    image_size=(224, 224),
    batch_size=32,
    color_mode='rgb'
)

# Load validation data
val_ds = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    class_names=class_names,
    validation_split=0.2,
    subset='validation',
    seed=42,
    image_size=(224, 224),
    batch_size=32,
    color_mode='rgb'
)

num_classes = len(class_names)

print("Number of classes:", num_classes)
print("Classes:", class_names)

# Preprocess images for VGG16
preprocess = tf.keras.applications.vgg16.preprocess_input

train_data = train_ds.map(
    lambda x, y: (preprocess(x), y)
)

val_data = val_ds.map(
    lambda x, y: (preprocess(x), y)
```

```
)

# c. Choose the pre-trained VGG16 model

base_model = VGG16(
    weights=None,
    include_top=False,
    input_shape=(224, 224, 3)
)

# Load downloaded VGG16 weights
base_model.load_weights(
    'vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5'
)

# d. Freeze base layers and add custom layers

base_model.trainable = False

inputs = Input(shape=(224, 224, 3))

x = base_model(inputs, training=False)
x = GlobalAveragePooling2D()(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.3)(x)

outputs = Dense(10, activation='softmax')(x)

model = Model(inputs, outputs)

# Compile the model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()

# e. Train the model

history = model.fit(
    train_data,
    validation_data=val_data,
    epochs=5
)

# f. Compute loss

loss, accuracy = model.evaluate(val_data)

print("Validation Loss:", loss)
print("Validation Accuracy:", accuracy * 100, "%")

# g. Evaluate the model
print("Model evaluation completed!")

# h. Visualize the predictions

for images, labels in val_ds.take(1):
```

```
    predictions = model.predict(
        preprocess(images), verbose=0
    )

    for i in range(3):

        predicted_class = np.argmax(predictions[i])
        actual_class = labels[i].numpy()

        plt.figure(figsize=(4, 4))
        plt.imshow(images[i].numpy().astype('uint8'))

        plt.title(
            "Predicted: " + class_names[predicted_class] +
            "\nActual: " + class_names[actual_class]
        )

        plt.axis('off')
        plt.show()

# i. Fine-tune the model

# Unfreeze VGG16 base model
base_model.trainable = True

# Freeze all layers except last 4
for layer in base_model.layers[:-4]:
    layer.trainable = False

# Compile with a small learning rate
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.00001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train again for fine-tuning
history_fine = model.fit(
    train_data,
    validation_data=val_data,
    epochs=3
)

# Final evaluation
final_loss, final_accuracy = model.evaluate(val_data)

print("Final Loss:", final_loss)
print("Final Accuracy:", final_accuracy * 100, "%")

# Save the model
model.save('fashion_mnist_vgg16_model.keras')

print("Model saved successfully!")
```

```
Found 12656 files belonging to 10 classes.
Using 10125 files for training.
Found 12656 files belonging to 10 classes.
Using 2531 files for validation.
Number of classes: 10
Classes: ['Ankle-boot', 'Bag', 'Coat', 'Dress', 'Pullover', 'Sandal', 'Shirt', 'S
neaker', 'T-shirt-top', 'Trouser']
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for
TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Pleas
e use WSL2 or the TensorFlow-DirectML plugin.
```

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0
vgg16 (Functional)	(None, 7, 7, 512)	14,714,688
global_average_pooling2d (GlobalAveragePooling2D)	(None, 512)	0
dense (Dense)	(None, 128)	65,664
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

Total params: 14,781,642 (56.39 MB)

Trainable params: 66,954 (261.54 KB)

Non-trainable params: 14,714,688 (56.13 MB)

```
Epoch 1/5
C:\Users\manis\AppData\Local\Python\pythoncore-3.12-64\Lib\site-packages\keras\sr
c\trainers\epoch_iterator.py:74: UserWarning: `shuffle=True` was passed, but will
be ignored since the data `x` was provided as a tf.data.Dataset. The Dataset is e
xpected to already be shuffled (via `.shuffle(buffer_size)`).
  self.data_adapter = data_adapters.get_data_adapter(
 52/317 ━━━━━━━━━━━━━━━━━━━━ 34:35 8s/step - accuracy: 0.4994 - loss: 1.7921
KeyboardInterrupt
```

In [ ]: